Noor Al-Quran App Workflow

Generated Technical Overview

April 10, 2026

1 Purpose

Noor Al-Quran is a Quran learning application with two main ideas:

- classical learning, where the user reads surahs and lessons from the Quran dataset;
- intelligent recitation support, where speech recognition and an NLP backend try to match the spoken Arabic text against Quran ayahs.

This document explains the application from the first page load to the final recitation result.

2 System Overview

The project is split into two parts:

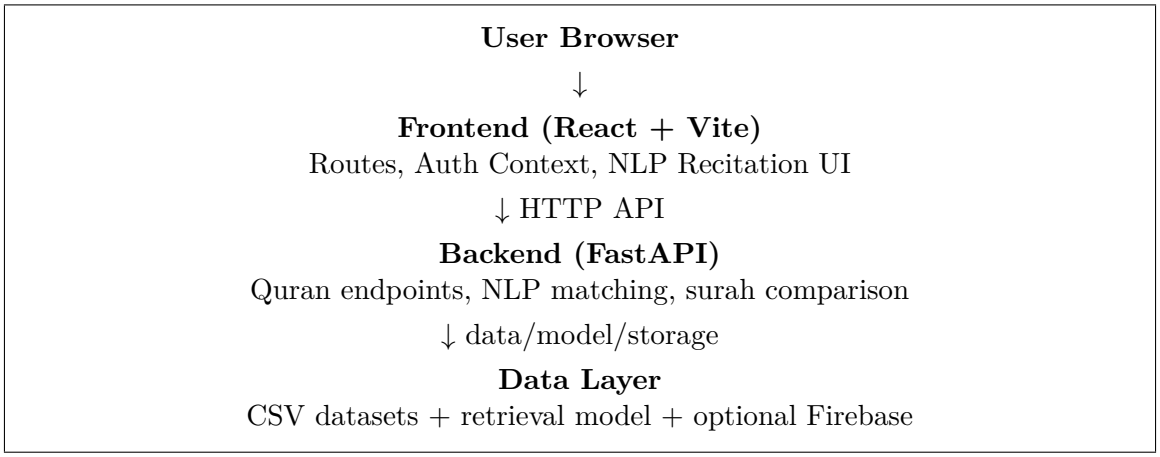
- **Frontend**: a React + Vite application inside `frontend/`.
- **Backend**: a FastAPI service inside `backend/` that serves Quran data, lesson data, audio, and NLP matching endpoints.

The frontend talks to the backend through the helper in `frontend/src/lib/api.ts`. The backend reads Quran datasets, loads or builds a recitation retrieval model, and exposes JSON endpoints that the UI consumes.

3 Schemas

This section adds quick visual schemas for the app architecture and runtime flow.

3.1 Global Architecture Schema



3.2 Recitation Session Schema

Start → Open `/nlp-reading` → Fetch surah list
→ Select surah → Fetch full ayahs → Show target ayah
→ Start microphone (`ar-SA`) → Build transcript
→ Local compare (normalize/tokenize/align/count mistakes)
→ Backend match (`/api/nlp/match-ayah`)
→ Decision: confirm / auto-advance / give hint
→ Last ayah? yes → finalize + save progress; no → next ayah

3.3 NLP Decision Schema

Input: transcript from speech recognition
→ **Step 1 (Frontend):** local text heuristics and mismatch highlights
→ **Step 2 (Backend):** TF-IDF + nearest-neighbor retrieval
→ **Step 3 (Frontend):** inspect top match + confidence
→ **Decision:**

- if surah and confidence are acceptable: accept ayah and move forward;
- if confidence is low or wrong surah: keep ayah and show hint;
- if transcript covers remaining surah: finish full-surah flow.

3.4 AI Model Internals Schema

Offline preparation (server startup)
Quran ayah text → Arabic normalization → TF-IDF vectorizer fit
→ ayah vectors + metadata index (surah, ayah number, original text)

Online inference (each transcript)
Transcript → normalization → transform with same TF-IDF model
→ nearest-neighbor search in ayah vector space
→ top-k candidates + similarity/confidence scores
→ frontend decision (accept, hint, or keep current ayah)

4 Start From Zero

When the app starts, the flow is roughly:

1. The browser loads the React entry point in `frontend/src/main.tsx`.
2. React renders `App` inside the root element.
3. `App.tsx` creates the router and wraps the app with the authentication provider.
4. The app lands on the splash or onboarding flow, then moves to login or signup if the user is not authenticated.
5. After authentication, protected pages such as Home, Reading, Lessons, Profile, Stats, and the NLP recitation page become available.

5 Frontend Entry Flow

The entry file is very small because its job is only to mount the app:

- `createRoot(...).render(<App />)` mounts the application.
- `App.tsx` defines all routes.
- `AuthProvider` makes the logged-in user available everywhere.
- `NotificationManager` keeps notifications available globally.

The route structure matters because the recitation page is not a standalone page; it is one part of a larger Quran learning journey.

6 Navigation and Protection

Most learning pages are wrapped with `ProtectedRoute`. That means:

- unauthenticated users are redirected away from private content;
- authenticated users can access reading, lessons, recitation, profile, and stats pages;
- the app keeps user progress and recitation attempts tied to an account when Firebase is enabled.

7 Where the NLP Recitation Page Fits

The page at `frontend/src/pages/NlpRecitation.tsx` is the center of the intelligent recitation feature. It is reachable at:

- `/nlp-reading`
- `/nlp-reading/:surahNo`

That page lets the user:

- pick a surah;
- open the microphone;
- speak Arabic recitation aloud;
- see the recognized transcript on screen;
- compare the spoken text with the expected ayah text;
- optionally enable auto-tilawa, which advances by NLP signals;
- save attempt history and surah completion data.

8 User Journey Inside the Recitation Page

Once the page opens, the following sequence happens.

8.1 1. Load Surah List

The page first requests the list of surahs from the backend through `api.getQuranSurahs()`. The response contains summary data such as:

- surah number;
- Arabic, English, and romanized names;
- total ayah count;
- place of revelation;
- whether the surah has sajdah ayahs.

If the user came from a URL like `/nlp-reading/2`, that surah is selected automatically. Otherwise, the first available surah is selected.

8.2 2. Load Full Surah Data

After a surah is selected, the page requests full surah details with `api.getQuranSurah(surahNo)`. That response includes every ayah in order, plus optional tafsir and metadata.

The page resets the session state at this point:

- current ayah goes back to 1;
- recognized text is cleared;
- mistake counters are reset;
- completion flags are cleared;
- any old NLP hints and mistake details are removed.

8.3 3. Show the Target Text

The UI displays the current ayah in Arabic script. If the current transcript contains the remaining part of the surah, the page can switch from single-ayah comparison to full-surah comparison.

That behavior is driven by the local helper that checks whether the spoken transcript contains the rest of the surah in order.

8.4 4. Start Speech Recognition

When the user presses the microphone button, the browser Speech Recognition API is opened in Arabic mode `ar-SA`. The page listens continuously and keeps interim results.

As speech arrives:

- the transcript is assembled from recognition events;
- the text is stored in local state;
- the screen updates in real time.

If the browser does not support speech recognition, the page shows an error and suggests a compatible browser.

8.5 5. Analyze the Spoken Transcript

When recognition stops, the page sends the final transcript to the backend NLP endpoint through `api.matchRecitedAyah(transcript, 3)`.

The backend returns the closest matching ayahs with confidence scores. The page uses those matches to:

- show likely ayahs in the NLP analysis panel;
- decide whether the recitation is close enough to advance;
- provide hint text when the model is unsure, the wrong surah is detected, or the confidence is too low.

8.6 6. Compare Spoken Words to Target Words

The page also performs local text comparison in the browser:

- Arabic diacritics are removed;
- punctuation is normalized;
- words are tokenized;
- the target text and spoken text are aligned;
- correct words and mistaken words are highlighted;
- an approximate mistake count is computed.

This gives immediate feedback even before the backend model is consulted.

8.7 7. Confirm Ayah or Advance Automatically

There are two ways the page advances:

- **Manual confirmation**: the user presses the confirm button after reciting an ayah.
- **Auto-tilawa**: the app watches the speech recognition transcript and uses NLP to move to the next ayah when the current ayah is judged complete.

If the transcript already contains the rest of the surah, the app can finish the full-surah flow immediately instead of advancing ayah by ayah.

9 NLP Decision Flow

The intelligent part of the app is a combination of local heuristics and backend matching.

9.1 Local Heuristics

The frontend has helpers to:

- normalize Arabic characters;
- tokenize Arabic words;
- align spoken words with expected words;
- count mistakes;
- extract sample mismatch pairs for display.

This helps the user see exactly where the spoken text diverges from the target ayah.

9.2 Backend Matching

The backend endpoint `/api/nlp/match-ayah` receives a transcript and returns top matching ayahs. The matching model is built from Quran text using text vectorization and nearest-neighbor search.

The page uses the highest-ranked match to determine whether:

- the model recognized the correct surah;
- the confidence is high enough;
- the user moved backwards or ahead in the surah;
- the current ayah should be accepted and counted.

10 Backend Workflow

The backend in `backend/main.py` is responsible for data and AI services.

10.1 Data Loading

At startup it tries to load:

- the Quran dataset;
- the tafsir dataset;
- the recitation retrieval model from disk;
- Firebase credentials and storage configuration if available.

If the saved recitation model is not available, the backend can build one in memory from the Quran dataframe.

10.2 Main Endpoints Used by the Recitation Page

The NLP page relies on these endpoints:

- `GET /api/quran/surahs` to fetch the surah list;
- `GET /api/quran/surah/{surahNo}` to fetch one surah with all ayahs;
- `POST /api/nlp/match-ayah` to find the closest ayah to a transcript;
- `POST /api/nlp/compare-surah` to verify whether a transcript covers the remaining part of a surah.

10.3 How the Matching Model Works

The backend normalizes Arabic text and converts each ayah into a character-level feature space using TF-IDF. Then it uses nearest-neighbor search to retrieve the closest Quran ayahs to the transcript.

This setup is suitable because recitation text is short, Arabic spelling variations are common, and diacritics are often missing in speech transcripts.

10.4 AI Model Step-by-Step (IA Engine)

The IA engine used in this app is a retrieval model, not a generative model. It does not create new text; it finds the closest existing ayahs.

1. **Build reference corpus**: every ayah is loaded from the Quran dataset with its surah and ayah identifiers.
2. **Normalize Arabic**: diacritics and text variants are normalized to reduce spelling/noise differences between recitation transcript and Quran text.
3. **Vectorize with TF-IDF**: each ayah is converted into a numeric vector using character-level TF-IDF so partial and noisy matches are still detectable.
4. **Index vectors**: all ayah vectors are stored in a nearest-neighbor index for fast lookup.
5. **Transform transcript**: each recognized speech transcript is normalized and transformed with the same fitted vectorizer.
6. **Retrieve top matches**: nearest-neighbor search returns top-k candidate ayahs with similarity/confidence values.
7. **Apply decision rules**: the frontend checks candidate surah, ayah position, and confidence thresholds to decide whether to advance, warn, or ask the user to retry.

In short, the model computes similarity in vector space: the closer the transcript vector is to an ayah vector, the higher the confidence for that ayah.

11 Persistence and Progress Tracking

The recitation page stores progress in two places.

11.1 Local Browser Storage

Completed surahs that qualify for the "heart" view are stored in browser local storage. A custom event keeps the UI in sync when the stored completion list changes.

This allows the app to remember completed surahs even before remote sync happens.

11.2 Firebase

If Firebase is configured, the app also writes user progress remotely:

- each ayah attempt is saved under `users/{uid}/ayahAttempts`;
- each completed surah is saved under `users/{uid}/surahCompletions`.

That means the app can preserve recitation history and completion summaries across sessions and devices.

12 End-of-Session Behavior

A session ends when the user reaches the last ayah or finishes a full-surah transcript.

At the end, the page:

- records the final mistake count;

- marks the session as finished;
- stops auto-tilawa if it was enabled;
- computes whether the surah qualifies for the heart view;
- optionally shows a detailed mistake breakdown by ayah.

If the final mistake count is under 10, the surah is added to the list of completed surahs shown in the heart section. If the count is 10 or more, the surah is still considered completed for the session, but it is not colored as a successful completion.

13 Other Pages in the App

Although the NLP recitation page is the most complex part, the rest of the app supports the learning journey:

- **Splash / Onboarding** introduces the app.
- **Login / Signup** handles authentication.
- **Home** acts as the dashboard.
- **Reading** shows Quran reading content.
- **Lessons** and **LessonDetail** support Tajweed lessons.
- **HeartPage** shows completed surahs.
- **Profile, Stats, Settings, Help** provide user support and configuration.

14 End-to-End Summary

The full flow of the application is:

1. the user opens the app in the browser;
2. React boots the frontend and loads authentication-aware routes;
3. the user signs in and reaches the learning pages;
4. the recitation page fetches surah metadata and full ayah text from the backend;
5. the user reads aloud into the microphone;
6. the browser converts speech to text;
7. the frontend compares the transcript locally and sends it to the backend NLP service;
8. the backend returns the best ayah matches or surah coverage result;
9. the page advances, counts mistakes, stores progress, and updates completion state;
10. when the surah is complete, the app records the result in local storage and, if available, in Firebase.

15 Files To Read First

If someone wants to inspect the implementation directly, the most important files are:

- `frontend/src/main.tsx`
- `frontend/src/App.tsx`
- `frontend/src/pages/NlpRecitation.tsx`
- `frontend/src/lib/api.ts`
- `frontend/src/lib/firebaseCollections.ts`
- `backend/main.py`